

Attendify: Advanced Facial Recognition Attendance System

Comprehensive System Architecture & Engineering Report

1. Executive Summary

Attendify is an enterprise-grade, full-stack facial recognition attendance ecosystem designed to seamlessly replace manual roll-call paradigms. It features an ultra-fast Python Machine Learning Microservice, a highly secure Node.js Database Engine, and a dynamic React.js Dashboard portal offering isolated data views for Administrators, Faculty, and Students.

By leveraging dynamic ML batching and cutting-edge Convolutional Neural Networks (CNNs) via InsightFace, the system achieves near realtime multi-face verification under heavy multi-tenant loads.

2. High-Level System Pipeline

The attendance pipeline is segmented into three decoupled computing environments:

- Frontend (React.js + Tailwind CSS + Vite):**
 - Provides a glass-morphic UI serving distinct Role-Based Access Control (RBAC) portals.
 - Teachers capture classroom photos and upload them directly via the browser. Active connections ping securely formatted JSON APIs.
 - REST API Backend (Node.js + Express + MongoDB):**
 - Acts as the central nervous system.
 - Handles JWT token rotation (`HttpOnly` cookies), hierarchical data locking (preventing students from seeing other students' data), and classroom lifecycle management.
 - Uses `multer` to intercept image streams and dispatches them internally to the ML microservice.
 - ML Microservice (Python + Flask + ONNX Runtime):**
 - A dedicated server running on port 5050 solely prioritizing deep-learning matrix multiplications.
 - Validates faces, performs geometric alignment, calculates embeddings, and fires the matched mathematical labels back to Node.js.
-

3. Server Designs & Architecture

3.1. The Node.js Express Core

The backend is mathematically strictly structured around an MVC (Model-View-Controller) topology hooked into a MongoDB backend. Route authorization is fiercely guarded using a two-step `protect` and `restrictTo(role)` middleware sequence.

- Cascading Integrity:** If an Admin deletes a classroom, the backend forcefully hunts down all orphaned `AttendanceSessions` and `AttendanceRecords` and scrubs them to prevent loose data memory leaks.
- Analytics Aggregation:** The backend calculates real-time attendance aggregates natively using `countDocuments` and timeline mapping algorithms depending on whether the consumer is a Teacher (global visibility) or a Student (isolated tracking).

3.2. Front-End Session Healing (Axios Interception)

Because JWT access tokens expire every 15 minutes, the React frontend utilizes an advanced Axios interceptor locking mechanism. If an API request hits a `401 Unauthorized`, the interceptor isolates the network queue, silences the crash, hits the `/refresh` token endpoint invisibly, recalculates the token, and replays the original backend request so fast the user literally never knows a token expired.

4. InsightFace Deep Learning: Internal Theory

InsightFace is an open-source 2D and 3D deep face analysis toolbox. Our system specifically runs the `buffalo_1` model suite entirely isolated on the CPU using the **ONNX (Open Neural Network Exchange)** Runtime.

InsightFace computes attendance via a two-stage neural pipeline:

Stage A: Detection (SCRFD)

1. **Sample and Scaled Frame Context:** Sample Frame is taken and rescaled down (to roughly 512x512) to cut down FLOPs.
2. **Bounding Box Isolation:** The **Sample and Computation of Receptive Field (SCRFD)** algorithm acts across the neural image mapping to detect raw boundaries defining where a face mathematically exists within a clustered classroom.
3. **Landmark Extraction:** Alongside the bounding box, it identifies 5 specific Key Points (Left Eye, Right Eye, Nose, Left Mouth, Right Mouth).

Stage B: Recognition (ResNet50 / ArcFace)

1. **Geometric Alignment (Affine Transform):** Using the 5 key points extracted above, the image is rotated and warped rigidly so every detected face guarantees the eyes and nose sit at the exact same mathematical coordinates before being fed into the heavy neural network.
 2. **ArcFace Embedding:** The aligned crop passes through an extensive ResNet50 Convolutional Neural Net. The network compresses the high-dimensionality image pixel data down into an un-corruptible sequence of **512 floating-point numbers** (a 512D Vector). This vector serves as a literal "digital fingerprint".
 3. **Additive Angular Margin Loss (ArcFace):** InsightFace is trained using ArcFace, which artificially maximizes the mathematical distance between different identities in the 512D hyperspace while compressing identical faces tighter together.
-

5. InsightFace Execution in Attendify

Out-of-the-box InsightFace processes faces iteratively. In Attendify, we engineered a **Dynamic Batching** architecture to explode processing speeds for massive classrooms.

1. **Pre-computation Initialization:** On startup, the Python server interfaces directly with the central image repository containing all registered student reference datasets. It computes the 512-Dimensional vector embedding mapping for every single known student, effectively loading the entire university identity matrix seamlessly into RAM before the first class even begins.
2. **Batched Inference:** When a teacher uploads a crowded classroom picture, instead of running ResNet50 iteratively on 60 isolated faces (which would stall the CPU), our server chunks all 60 aligned face crops into a single contiguous multi-dimensional Numpy Tensor `(N, 3, 112, 112)`.
3. **Hardware Acceleration:** The Matrix multiplication runs sequentially mapped across all parallel CPU threads in one giant burst through the ONNX execution provider.
4. **Vectorized Cosine Similarity:** Finally, the system executes a vectorized mathematical "Dot Product" matrix multiplication comparing the classroom detected embeddings against the RAM loaded RAM matrix. The system generates normalized `sim_matrix` scores.
5. **Threshold Bounding:** Any bounding box returning a vector similarity higher than `0.35` against a stored identity is dynamically locked as a match (e.g. `Saksham Chhabra`) and relayed backwards through to the

Node runtime.

6. Dynamic Roster Verification

To seal the pipeline loop, the matched labels are shuttled out of Python and back to the React UI stream (`TakeAttendance.jsx`). The UI natively extracts the mathematical payload string, dynamically parses it via substring validations enforcing `lowercase` mappings against the student's `name` or `rollNumber` , and dynamically flips their checkbox to `Present` while painting a styled bounding-box directly over the DOM reference image.

The instructor then validates the AI suggestions and commits them rigidly to the secure database array.